

PROJECT IMPULSE

Impulse

Single-Cycle CPU Simulation Report

Vivado 2025.1 xsim behavioral regression | CPI = 1

DOCUMENT	Impulse Single-Cycle Simulation Report
PROJECT	Impulse (Single-Cycle CPU)
IMPLEMENTATION	Single-Cycle
SIMULATOR	Vivado 2025.1 xsim
REPORT DATE	June 2026
SOURCE PDF	ARIA-32_SingleCycle_SimReport.pdf

1

Simulation Summary

13 / 13 testbenches
PASS
STATUS

Single-Cycle
IMPLEMENTATION

1
CPI

Vivado 2025.1 xsim
SIMULATOR

This report summarizes the behavioral simulation regression for the Impulse single-cycle CPU. All checks compare expected DMEM contents and register values after program halt.

Single-Cycle Datapath Flow

FETCH
IMEM + PC

Read instruction at current PC and calculate next PC.

DECODE
Control + Reg File

Decode opcode, condition, operands, and register sources.

EXECUTE
ALU / Branch

Run arithmetic, logic, compare, branch, and address operations.

MEMORY
DMEM

Perform LDR/STR access for word-addressed data memory.

WRITEBACK
RF + PC

Commit register result, PC update, and interrupt return state.

All blocks are evaluated in one clock period; architectural state commits on the clock edge.

Regression Status Matrix

Testbench	Scope	Timeout	Approx Cycles	Status
tb_cpu	ISA integration	500 cycles	~85	PASS
tb_memcpy	10-word memory copy loop	500 cycles	~66	PASS
tb_array_sum	$C[i] = A[i] + B[i]$, $N=25$	700 cycles	~230	PASS
tb_minmax	Min/max linear scan, $N=25$	600 cycles	~190	PASS
tb_sort	Bubble sort ascending, $N=25$	3000 cycles	~1800	PASS
tb_factorial	$0!$ through $12!$ via BL/BX	1200 cycles	~750	PASS
tb_bitops	AND/OR/XOR/NOT/LSL/LSR x 8 inputs	500 cycles	~160	PASS
tb_gcd	Euclidean GCD x 6 pairs	1000 cycles	~600	PASS
tb_power	$base^{exp}$ binary exponentiation x 8	800 cycles	~450	PASS
tb_isqrt	$\text{floor}(\text{sqrt}(n)) \times 10$	2500 cycles	~1500	PASS
tb_collatz	Collatz steps-to-1 x 8	3000 cycles	~1800	PASS
tb_fibonacci	$F(n)$ iterative x 8	800 cycles	~480	PASS
tb_bsearch	Binary search 16 queries / 150 elements	5000 cycles	~2800	PASS

2 RTL Source Files

The single-cycle implementation uses a direct datapath with combinational decode, ALU, memory control, register writeback, and interrupt handling in `cpu_top.v`.

RTL File	Role in Simulation
<code>cpu_top.v</code>	Top-level: datapath, APB slave, interrupt controller
<code>control.v</code>	Instruction decode to control signals
<code>alu.v</code>	32-bit ALU: ADD/SUB/MUL/AND/OR/XOR/NOT/LSL/LSR/CMP
<code>reg_file.v</code>	16x32 register file (R0-R15; SP=R13, LR=R14; flat signals <code>r0..r15</code>)
<code>cond_check.v</code>	Condition code evaluator (AL/EQ/NE/GT/LT/GE/LE)
<code>inst_mem.v</code>	256x32 instruction SRAM, APB write-loaded at reset
<code>data_mem.v</code>	256x32 data SRAM
<code>defines.v</code>	ISA constants: format codes, opcodes, condition codes (<code>`include</code>)

3 Architecture and Testbench Strategy

Single-Cycle Architecture

Area	Behavior
Implementation	Single-cycle: all pipeline stages execute within one clock period
CPI	1
PC update	Every clock at the end of the combinational datapath
Pipeline registers	None
Register file	16x32 flat signals <code>r0..r15</code> ; SP=R13, LR=R14
IMEM	256x32 APB-loaded at reset (<code>prst_n=1</code> , <code>rst_n=0</code> during load)
DMEM	256x32 word-addressed; write-enabled every clock on STR
Interrupts	IRQ/FIQ lines; <code>MODE_SYS/MODE_IRQ/MODE_FIQ</code> ; RETI returns
Halt detection	Two consecutive identical PC values at any posedge clk

Simulation Method

Area	Method
Program loading	APB writes to IMEM with <code>rst_n=0</code> and <code>prst_n=1</code> ; then <code>rst_n=1</code> starts CPU execution
Halt detection	Sample <code>dut.pc</code> every posedge clk; halt when <code>pc_curr == pc_prev</code>
Simulation end	<code>\$stop</code> after <code>verify()</code> ; xsim stays alive for <code>close_simulation</code>
Timeout	<code>\$stop</code> triggered after each testbench-specific watchdog cycle count
VCD dump	<code>\$dumpfile</code> / <code>\$dumpvars</code> at module start for waveform inspection
Result check	Display PASS/FAIL per expected DMEM word or register value
<code>\$stop</code> vs <code>\$finish</code>	All testbenches use <code>\$stop</code> so Vivado <code>close_simulation</code> can release <code>simulate.log</code> before the next TB launches in <code>run_all.tcl</code>

4 Detailed Testbench Results

Each testbench loads its program through APB, releases reset, waits for halt or timeout, and verifies expected architectural state.

01 tb_cpu

PASS

Source	tb/tb_cpu.sv	Timeout	500 cycles
Cycles	~85	Halt	Two consecutive identical PC values at any posedge clk.

General ISA integration test. Exercises all major instruction categories in one straight-line program: arithmetic, bitwise, branch taken/not-taken, load/store round-trip, INC/DEC, PUSH/POP, BL/BX subroutine call-return, and IRQ vector table (RETI).

INSTRUCTION COVERAGE

MOVI ADD R-type SUB R-type MUL R-type AND OR XOR NOT LSL LSR CMP BGT BNE STR LDR INC
DEC PUSH POP BL BX B AL RETI

PASS CRITERIA

- R2 = 15 (13 from ADD R0+R1, then +2 INC inside sub_add2 subroutine)
- R5 = 3055 (0xBEF from MOVI after BGT taken; FAIL path sets R5=0)
- R6 = 3055 (LDR from DMEM[R0]; load/store round-trip verify)
- R15 = 3055 (POP'd from stack; must equal R5)
- SP = SP_INIT (stack balanced after PUSH/POP pair)
- R0 = 10, R1 = 3, R3 = 7, R4 = 30, R7 = 1, R8 = 15, R9 = 14, R11 = 24, R12 = 3

NOTES

- IRQ pulsed at cycle ~100 in SYS mode; IRQ handler is a single RETI.

02 tb_memcpy

PASS

Source	tb/tb_memcpy.sv	Timeout	500 cycles
Cycles	~66	Halt	Counter reaches 0 and PC stabilizes.

Memory copy loop copies 10 words from DMEM[0x100..0x124] to DMEM[0x200..0x224].

INSTRUCTION COVERAGE

MOVI LDR STR ADD immediate offset DEC BNE

PASS CRITERIA

- DMEM[0x200..0x224] matches source for all 10 words
- R0 = 0x128, R1 = 0x228, R2 = 0

NOTES

- Source words are preloaded; 10 LDR+STR pairs are verified.

03 tb_array_sum**PASS**

Source	tb/tb_array_sum.sv	Timeout	700 cycles
Cycles	~230	Halt	Loop completes after N=25 elements and PC stabilizes.

Vector addition computes $C[i] = A[i] + B[i]$ for $N=25$ elements.

INSTRUCTION COVERAGE

MOVI LDR ADD R-type STR INC CMP BNE

PASS CRITERIA

- DMEM[192..288] contains all $C[i] = A[i] + B[i]$ results
- Example: $A[3]=951 + B[3]=629 = 1580$ at DMEM[204]
- Example: $A[22]=743 + B[22]=854 = 1597$ at DMEM[280]

04 tb_minmax**PASS**

Source	tb/tb_minmax.sv	Timeout	600 cycles
Cycles	~190	Halt	Loop completes after N=25 elements and PC stabilizes.

Linear scan finds the minimum and maximum values across a 25-element array.

INSTRUCTION COVERAGE

MOVI LDR CMP BLT BGT STR INC BNE

PASS CRITERIA

- DMEM[200] = 18
- DMEM[201] = 951

NOTES

- Array values include {427,83,612,951,38,776,245,18,534,889,...,667}.

05 tb_sort**PASS**

Source	tb/tb_sort.sv	Timeout	3000 cycles
Cycles	~1800	Halt	Outer and inner bubble-sort loops complete and PC stabilizes.

Bubble sort orders 25 words in ascending order.

INSTRUCTION COVERAGE

MOVI LDR CMP BLE STR INC DEC BNE ADD

PASS CRITERIA

- DMEM[64..160] sorted ascending
- Expected order begins [18, 29, 38, 57, 83] and ends with 951

NOTES

- Complexity: $N*(N-1)/2 = 300$ worst-case compare-swap iterations.
- Worst case is about 300 swaps x ~8 instructions = ~2400 instructions.

06 tb_factorial**PASS**

Source	tb/tb_factorial.sv	Timeout	1200 cycles
Cycles	~750	Halt	Factorial sequence completes and PC stabilizes.

Iterative factorial test computes $n=0..12$ through a BL/BX subroutine flow.

INSTRUCTION COVERAGE

MOVI MUL STR BL BX R14 PUSH POP TST BEQ DEC BNE

PASS CRITERIA

- $DMEM[128] = 1$ (0!)
- $DMEM[129] = 1$, $DMEM[130] = 2$, $DMEM[131] = 6$, $DMEM[132] = 24$
- $DMEM[133] = 120$, $DMEM[134] = 720$, $DMEM[140] = 479001600$ (12!)

NOTES

- BL/BX uses LR for subroutine return.

07 tb_bitops**PASS**

Source	tb/tb_bitops.sv	Timeout	500 cycles
Cycles	~160	Halt	All eight input groups complete and PC stabilizes.

Bitwise regression runs AND, OR, XOR, NOT, LSL, and LSR over eight input values, producing 48 outputs.

INSTRUCTION COVERAGE

MOVI LDR AND OR XOR NOT LSLI LSRI STR INC CMP BNE

PASS CRITERIA

- For each input: $AND\ val \& 0x0F$, $OR\ val | 0xF0$, $XOR\ val \wedge 0xFF$, $NOT\ \sim val$
- For each input: $LSL\ val \ll 4$ and $LSR\ val \gg 4$
- 48 result words are written at $DMEM[128..303]$

NOTES

- Each of eight inputs produces six consecutive results.

08 tb_gcd**PASS**

Source	tb/tb_gcd.sv	Timeout	1000 cycles
Cycles	~600	Halt	All six GCD pairs complete and PC stabilizes.

Euclidean subtraction GCD is evaluated across six input pairs using BL/BX subroutine return.

INSTRUCTION COVERAGE

MOVI LDR SUB R-type CMP BEQ BGT BL BX R14 PUSH POP STR

PASS CRITERIA

- $DMEM[128] = 6$, $DMEM[129] = 21$, $DMEM[130] = 25$
- $DMEM[131] = 120$, $DMEM[132] = 1$, $DMEM[133] = 36$

09 tb_power**PASS**

Source	tb/tb_power.sv	Timeout	800 cycles
Cycles	~450	Halt	All eight exponentiation cases complete and PC stabilizes.

Binary exponentiation evaluates eight base/exponent pairs.

INSTRUCTION COVERAGE

MOVI LDR MUL LSR TST BEQ BNE STR INC

PASS CRITERIA

- DMEM[128] = 1048576, DMEM[129] = 2187, DMEM[130] = 15625, DMEM[131] = 16807
- DMEM[132] = 1296, DMEM[133] = 27, DMEM[134] = 4, DMEM[135] = 1

10 tb_isqrt**PASS**

Source	tb/tb_isqrt.sv	Timeout	2500 cycles
Cycles	~1500	Halt	All ten square-root cases complete and PC stabilizes.

Integer square root calculates floor(sqrt(n)) for ten values using binary search.

INSTRUCTION COVERAGE

MOVI LDR ADD SUB LSR MUL CMP BGT BLT BEQ STR

PASS CRITERIA

- DMEM[128] = 0, DMEM[129] = 1, DMEM[130] = 1, DMEM[131] = 3, DMEM[132] = 4
- DMEM[133] = 10, DMEM[134] = 15, DMEM[135] = 100, DMEM[136] = 351, DMEM[137] = 1000

NOTES

- Binary search in $[0, n/2]$; high timeout covers the 1000000 case.

11 tb_collatz**PASS**

Source	tb/tb_collatz.sv	Timeout	3000 cycles
Cycles	~1800	Halt	All eight Collatz cases complete and PC stabilizes.

Collatz regression counts steps to convergence for eight input values.

INSTRUCTION COVERAGE

MOVI LDR TST BEQ LSR MUL ADD INC STR BNE

PASS CRITERIA

- DMEM[128] = 0, DMEM[129] = 1, DMEM[130] = 7, DMEM[131] = 8
- DMEM[132] = 16, DMEM[133] = 14, DMEM[134] = 111, DMEM[135] = 25

NOTES

- DMEM[131] = 8 corresponds to $n=4$: 4 to 2 to 1 then back, 8 steps to convergence.

12 tb_fibonacci

PASS

Source	tb/tb_fibonacci.sv	Timeout	800 cycles
Cycles	~480	Halt	All eight Fibonacci cases complete and PC stabilizes.

Iterative subroutine computes eight Fibonacci values.

INSTRUCTION COVERAGE

MOVI LDR ADD R-type BL BX R14 PUSH POP CMP BEQ BNE STR

PASS CRITERIA

- DMEM[128] = 0, DMEM[129] = 1, DMEM[130] = 1, DMEM[131] = 5
- DMEM[132] = 21, DMEM[133] = 55, DMEM[134] = 377, DMEM[135] = 4181

13 tb_bsearch

PASS

Source	tb/tb_bsearch.sv	Timeout	5000 cycles
Cycles	~2800	Halt	All 16 search queries complete and PC stabilizes.

Binary search over 150 sorted elements handles 16 queries: 12 hits and 4 misses.

INSTRUCTION COVERAGE

MOVI LDR indexed ADD SUB LSR mid MUL byte addr CMP BEQ BGT BLT STR BL BX R14

PASS CRITERIA

- DMEM[232..243] contains found indices {0,14,29,30,50,79,80,100,119,120,135,149}
- DMEM[244..247] contains 0xFFFFFFFF x4 for missing queries
- Total 16 queries: 12 hits + 4 misses

5 Regression Summary

The regression suite covers ISA integration, memory movement, array arithmetic, control flow, subroutines, arithmetic kernels, and search workloads.

Testbench	Description	Approx Cycles	Status
tb_cpu	ISA integration	~85	PASS
tb_memcpy	10-word memory copy loop	~66	PASS
tb_array_sum	$C[i] = A[i] + B[i]$, $N=25$	~230	PASS
tb_minmax	Min/max linear scan, $N=25$	~190	PASS
tb_sort	Bubble sort ascending, $N=25$	~1800	PASS
tb_factorial	$0!$ through $12!$ via BL/BX	~750	PASS
tb_bitops	AND/OR/XOR/NOT/LSL/LSR x 8 inputs	~160	PASS
tb_gcd	Euclidean GCD x 6 pairs	~600	PASS
tb_power	base^{exp} binary exponentiation x 8	~450	PASS
tb_isqrt	$\text{floor}(\text{sqrt}(n)) \times 10$	~1500	PASS

Testbench	Description	Approx Cycles	Status
tb_collatz	Collatz steps-to-1 x 8	~1800	PASS
tb_fibonacci	F(n) iterative x 8	~480	PASS
tb_bsearch	Binary search 16 queries / 150 elements	~2800	PASS

13 / 13 testbenches PASS

All checks were verified against expected DMEM contents and register values. Cycle counts are estimates and depend on program length and branch outcomes.